

Q.1

$$\text{CPU Time} = \text{IC}_A * \text{CPI} * T_c$$

$$\begin{aligned}\text{IC}_A &= \text{CPU Time}_A / \text{CPI} * T_c \\ &= 1 / (1 * 1 / (4 * 10^9)) \\ &= 4 * 10^9\end{aligned}$$

$$\begin{aligned}\text{IC}_B &= \text{CPU Time}_B / \text{CPI} * T_c \\ &= 1 / (1.25 * 1 / (4 * 10^9)) \\ &= 3.2 * 10^9\end{aligned}$$

In the new processor

$$\begin{aligned}\text{CPU Time}_A &= \text{IC}_A * \text{CPI} * T_c \\ &= 4 * 10^9 * 0.6 * 0.33 * 10^{-9} \\ &= 0.792 \text{ sec}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{IC}_A * \text{CPI} * T_c \\ &= 3.2 * 10^9 * 1 * 0.33 * 10^{-9} \\ &= 1.056 \text{ sec}\end{aligned}$$

For program A our processor is faster.

For program B our friend's processor is faster.

Q2.

a)

The code is extracting the Least Significant Byte (1st Byte from right) of a given 32 bit value stored in a0 and stores it in v0.

b)

```
add $t0, $zero, $zero // i = 0
addi $t3, $gp, 4      // t3 = &i
```

```

addi $t1, $gp, 8      // t1 = &a[0] or t = a
loop :
    sll $t2, $t0, 5    // t2 = 32 * t0 or 32 * i
    sw $t2, 0($t1)     // a[i] = 32 * i
    add $t0, $t0, 1    // i = i + 1
    add $t1, $t1, 4    // t1 = a + 4
    bne $t0, 10, loop // if i != 10 goto loop
sw $t0, 0($t3)        // update final value of i in memory

```

c)

```

sll $s1, $s0, 12
srl $s1, $s1, 12

```

Question 3

Q3 solution:

Converting IEEE 754 single precision floating point registers into decimal numbers

number 1: 010000110010100...

sign = 0, Ebias = 10000110 = 134, mantiaa(M) = 010100 ...

DECIMAL VALUE = $(-1)^{\text{SIGN}} \times (1.M) \times 2^{(\text{Ebias} - \text{bias})}$, bias = 127
= 1.0101×2^7 or 168

number 2: 010000100110000...

sign = 0, Ebias = 10000100 = 132, mantiaa(M) = 110000 ...

DECIMAL VALUE = $(-1)^{\text{SIGN}} \times (1.M) \times 2^{(\text{Ebias} - \text{bias})}$, bias = 127
= 1.1100×2^5 or 56

Adding the Following number in Binary,

As per algorithm, convert the smaller exponent number to larger exponent number

0 10000110 01010 --> 1.0101×2^7

0 10000100 11000 --> 0.0111×2^7

 $1.1100 \times 2^7 = 224$

Register will store the bits = 0 10000110 110000....

Question 4

Instruction	Instruction fetch + PC Increment	Register read + Branch target computation	Execute	Memory read	Register write back
lw \$t1, 100(\$t2)	PCWriteCond = PCWrite = 1 IorD = 0 MemRead = 1 MemWrite = IRWrite = 1 RegDst = MemtoReg = RegWrite =	PCWriteCond = PCWrite = IorD = MemRead = MemWrite = IRWrite = RegDst = MemtoReg = RegWrite =	PCWriteCond = PCWrite = IorD = MemRead = MemWrite = IRWrite = RegDst = MemtoReg = RegWrite = ALUSrcA = 1 ALUSrcB = 10 ALUop = Add PCSource =	PCWriteCond = PCWrite = IorD = 1 MemRead = 1 MemWrite = IRWrite = RegDst = MemtoReg = RegWrite = ALUSrcA = ALUSrcB = ALUop = PCSource =	PCWriteCond = PCWrite = IorD = MemRead = MemWrite = IRWrite = RegDst = 0 MemtoReg = 1 RegWrite = 1 ALUSrcA = ALUSrcB = ALUop = PCSource =
beq \$s1, \$s2, 25	ALUSrcA = 0 ALUSrcB = 01 ALUop = Add PCSource = 0	ALUSrcA = 0 ALUSrcB = 11 ALUop = Add PCSource =	PCWriteCond = 1 PCWrite = 0 IorD = MemRead = MemWrite = IRWrite = RegDst = MemtoReg = RegWrite = ALUSrcA = 1 ALUSrcB = 00 ALUop = Sub PCSource = 1	NA	

Question 5

opcode	rs	rt	rd	ru(modified)	function
[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]

	Instruction Fetch + PC Increment	Register Read + Branch target computation	Execute1	Execute2	Register write back
PCWriteCond	X	X	X	X	X
PCWrite	1	X	X	X	X
IorD	0	X	X	X	X
MemRead	1	X	X	X	X
MemWrite	X	X	X	X	X
IRWrite	1	X	X	X	X
RegDst	X	X	X	X	1
MemtoReg	X	X	X	X	0
RegWrite	X	X	X	X	1
ALUSrcA	00	00	01	10	X
ALUSrcB	01	11	00	00	X
ALUOp	Add	Add	Add	Add	X
PCSource	0	X	X	X	X
RegRead_mux	X	0	0	1	X

RegRead_mux

ALUSrc A become 2 bits

